

How well does the AI coach plan the next round?

Thirty-two simulated learner sessions put through the live between-level curriculum planner, measured against the deterministic fallback planner on identical inputs and checked item-by-item by an independent solver from a different provider family.

PLANNER MODEL	INDEPENDENT SOLVER	SESSIONS	BILLED COST
google/gemini-3.1-pro-preview	openai/gpt-5.6-terra	32 (8 profiles × 4)	\$5.98
RUN DATE			
2026-09-04			

1 Summary

PLAN SERVED 100% 32 of 32 sessions	ACCEPTED FIRST TRY 100% no repair retry needed	MEDIAN LATENCY 89_s p95 195s
COST PER PLAN \$0.173 per served plan	SOLVER AGREEMENT 99.6% 956/960 • control 100%	

The coach returned a usable plan for every session, all of them accepted on the first attempt with no repair retry, so the repair loop and the deterministic fallback were never exercised. An independent solver from a different provider family agreed with 99.6% of 960 generated answer keys, against 100.0% on the human-authored control. Against the deterministic fallback it wins decisively where the fallback structurally cannot compete — novel material (99% of stems are new), diagnostic distractor labels (100% specific rather than generic), and questions that actually sit on the stated curriculum (65% against 17%). It is beaten on concept targeting, where the fallback scores 1.00 by copying the report’s own weak-concept list. The costs are 89s median latency (p95 195s) and \$0.17 per plan — affordable between rounds, far too slow inside one.

2 System under test

What actually runs when a round ends.

Punch English is a four-lane rhythm-boxing game for learning English. The live round is driven by a deterministic adaptive algorithm; the language model is used only between rounds. When a round ends the game posts a *learner report* to a local endpoint (`/api/punchkt/plan`) and asks for one thing back: a plan for the next round — end-of-round feedback, a curriculum, and a pool of exactly 30 multiple-choice questions.

The learner report carries three kinds of evidence, and the prompt requires all three to be used: per-concept mastery and uncertainty; the specific wrong options a player keeps choosing; and which concepts are due for review under a forgetting curve. It also separates motor evidence from language evidence — a punch that never lands is not an English mistake.

The response is constrained by a JSON schema and then checked twice on our side: the schema parse enforces the shape and the 30-question count, and `findPlanIssues` enforces the semantics — unique question stems, no duplicate options, a misconception label on every wrong option and the literal `"none"` on the correct one, and a difficulty mix that both totals 30 and matches the questions actually written. A plan that fails either check is sent back once with the list of faults; if the retry also fails, the game silently falls back to a deterministic local planner. That fallback is the baseline this report measures against.

3 Method

Where the inputs come from and what counts as evidence.

3.1 Simulated learners

Real play data does not exist at the volume an evaluation needs, so the harness synthesises it. Eight learner profiles are defined by their measurable behaviour — a cold start, a struggling beginner, a player with specific grammar gaps, an advanced player, one whose punches keep missing, one who punches early, one returning after a long gap with decayed retention, and one repeating the same misconception. Each profile is simulated 4 times with different seeds, giving 32 independent sessions. A single fixed clock is used for the whole batch so every simulated report is reproducible.

The profiles are checked before any billed call: a free phase prints each one's realised accuracy, miss rate and early-punch rate against its target, so a broken generator is caught before it can be mistaken for a broken model.

3.2 The comparison

Every metric below is computed twice — once on the model's plan, once on the deterministic fallback planner's plan for the *same* learner report — by the same code. That pairing is the point: it answers whether the model earns its cost and latency, rather than whether its output looks plausible in isolation.

3.3 Independent item check

Question quality cannot be graded by the model that wrote the questions. A separate solver from a **different provider family** answers every generated item without being shown the answer key, and reports its own concern about each item (ambiguous stem, more than one defensible option, no correct option, overlapping options). Agreement between that solver and the plan's key is evidence about the item.

The same solver also answers the 30 **human-authored** questions already shipped in the game. Those are the control: whatever the solver gets "wrong" on hand-written items is the solver's own error rate, and it sets the floor the generated items should be read against. Disagreements are candidates for review, not verdicts — the harness writes them to `disagreements.json` for adjudication by hand.

4 What is measured

Six groups. Everything except the solver check is computed without a model and without a human, so it can be re-run on any future prompt or model change and compared directly.

Reliability

Whether a usable plan comes back at all: schema-valid rate, acceptance on the first attempt, recovery via the repair retry, and the served rate (no fallback). Reported with Wilson intervals, which stay sensible at rates near 0 and 1. Latency, token usage and billed cost sit here too.

Item soundness

Independent-solver agreement with the answer key, split by difficulty, read against the human-authored control, plus the solver's own concern categories.

Curriculum targeting

Whether the plan addresses what the learner actually needs: precision of the chosen target concepts against the union of weak, uncertain and due concepts (exact and fuzzy name matching, since concept names travel as free text); recall of weak and due concepts; misconception coverage; and the share of questions that sit on the stated curriculum.

Item-writing quality

The biases that make a question bank easy to game: answer-position balance (χ^2 against uniform), the correct answer being the longest or shortest option, duplicate and near-duplicate stems, and whether distractor labels actually diagnose anything rather than restating "wrong".

Player-facing language

Everything the learner reads is checked for reading grade (Flesch–Kincaid; the prompt targets a twelve-year-old) and for leakage of internal or learning-science vocabulary the prompt bans — "mastery", "misconception", "adaptive", "LLM" and the rest.

Novelty

Whether each session gets fresh material: the share of stems not already in the authored bank, and pairwise stem overlap between plans, which is what catches a model quietly reusing the same thirty questions every round.

5 Results

5.1 Reliability

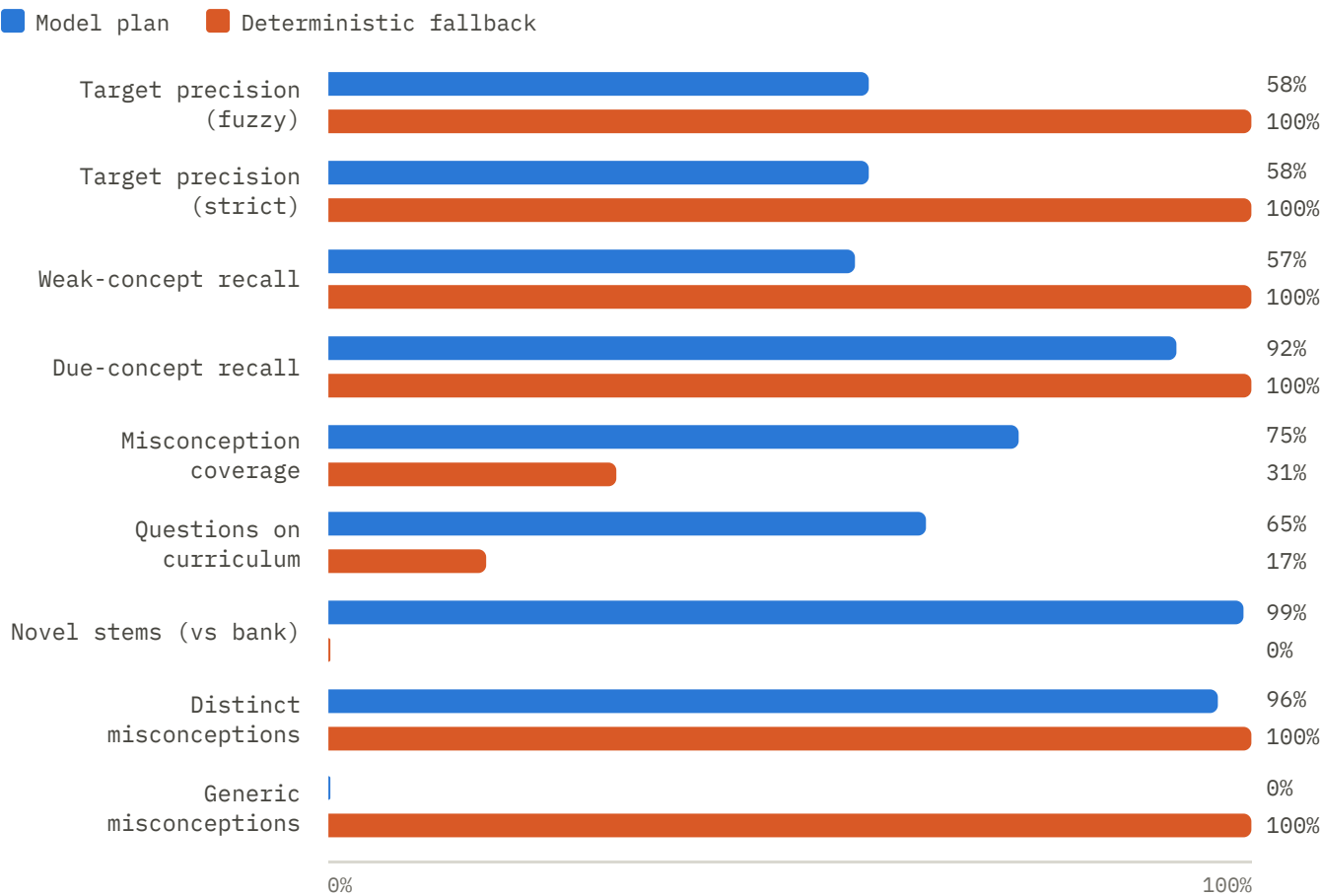
ATTEMPT FUNNEL



MEASURE	VALUE	95% INTERVAL
Plan served (no fallback)	32/32 = 100.0%	[89%-100%]
Accepted on first attempt	32/32 = 100.0%	[89%-100%]
Schema-valid attempts	32/32 = 100.0%	[89%-100%]
Recovered by the repair retry	0/0 retries	–
Latency mean / p50 / p95 / max	99.8 / 88.6 / 194.7 / 200.4 s	–
Tokens in / out / reasoning	5854 / 13550 / 6523	–
Cost total / per served plan	\$5.5491 / \$0.1734	–

No attempt that reached the API was rejected by either validation stage.

5.2 Plan quality against the deterministic baseline

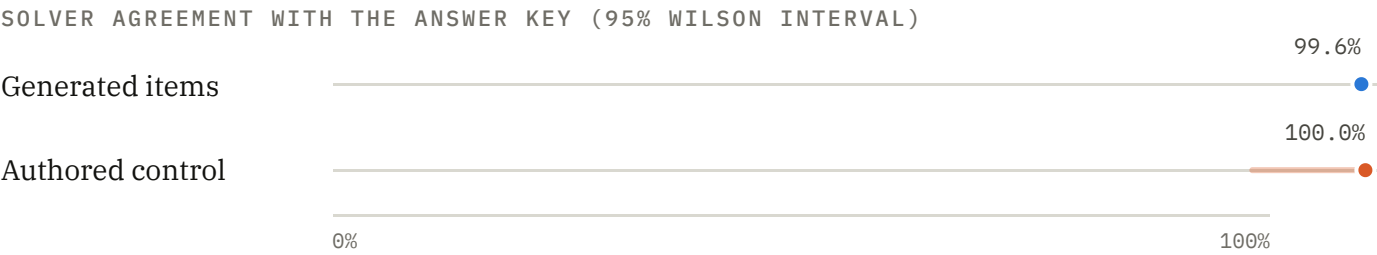


Target precision and weak/due recall are **1.00 by construction** for the deterministic planner — it copies its targets straight out of the report — so those four rows are not a contest. The rows that separate the two are novelty, misconception quality and whether the questions actually sit on the curriculum the plan claims.

NON-RATE MEASURES

MEASURE	MODEL PLAN	DETERMINISTIC
Reading grade (Flesch-Kincaid)	7.13 ±0.80	5.30 ±0.38
Categories used (of 6)	5.78 ±0.49	6.00 ±0.00
Near-duplicate stem pairs	0.06 ±0.35	0.00 ±0.00
Mean stem length (words)	9.39 ±1.56	6.10 ±0.00
Answer-position χ^2	0.19 ±0.15	27.07 ±0.00
Correct answer at slot 3	7.03 ±0.40	0.00 ±0.00
Longest option is correct	8.28 ±2.95	10.00 ±0.00
Plans with balanced positions	32/32	0/32

5.3 Item soundness



ITEM SET	AGREED	RATE	CONCERNS RAISED
Generated – all	956/960	99.6%	14
Generated – easy	320/320	100.0%	–
Generated – medium	318/320	99.4%	–
Generated – hard	318/320	99.4%	–
Human-authored control	30/30	100.0%	1

The control matters more than the headline: the solver agreed with 100.0% of the *human-authored* bank, so that is the ceiling this method can measure, not 100%. Generated items sit at 99.6%. Concerns the solver raised on generated items: `multiple-correct` × 12, `no-correct` × 1, `ambiguous-stem` × 1. Every disagreement is written to `disagreements.json` for review by hand before it is called a fault.

5.3.1 Every disagreement, adjudicated

Small enough to check by hand, so here they all are – every item where the solver’s answer differed from the plan’s key. The solver’s pick is marked `solver` and the plan’s key `key`. A further 11 items were answered correctly but still flagged with a concern.

R20-P5-MOTOR-MISSES · ITEM 23 · HARD · GRAMMAR · FLAGGED MULTIPLE-CORRECT

___ you need help, please call me.

0 Should

1 If

2 Would

3 Had

key

solver

Choose the correct imaginary situation.

0 Had I time, I would read it.

solver

1 Were I to have time, I would read it.

key

2 Have I time, I will read it.

3 Was I having time, I read it.

Never before ____ felt so confident.

0 he had

1 he has

2 had he

key

3 has he

solver

Rarely ____ so happy to see someone.

0 he is

1 is he

solver

2 he was

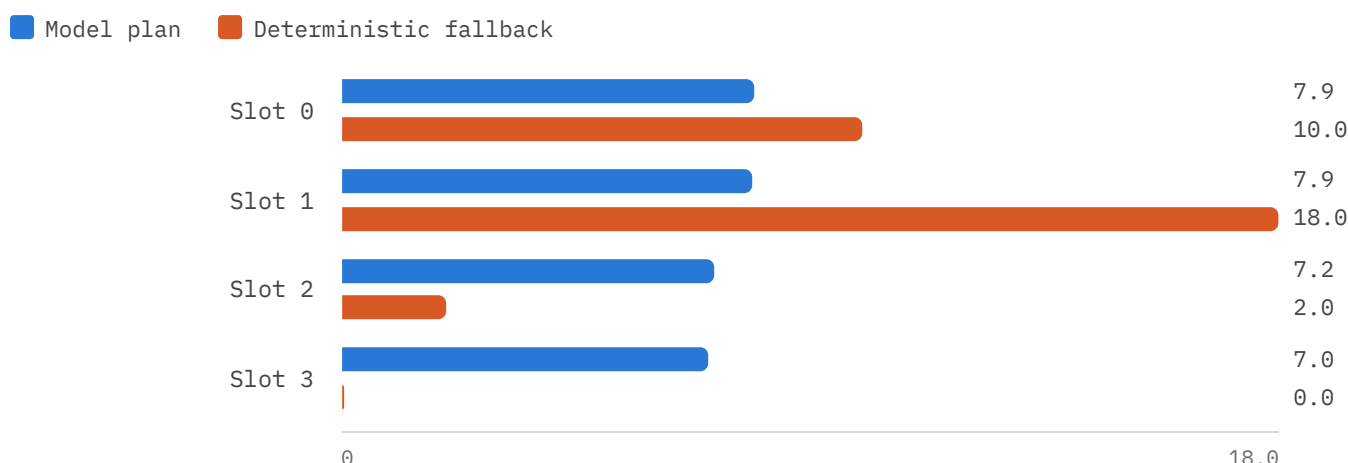
3 was he

key

FINDING

All 4 disagreements are the same defect, and the solver is right in every case. Each is a negative-adverb inversion or a conditional whose stem supplies no tense or register anchor, so two options are simultaneously correct: “Never before *had he* / *has he* felt so confident” are both good English, and so are “*Should* you need help” and “*If* you need help”. The model is not making grammar errors — it is writing items whose stems under-determine the answer. That is a narrow, fixable prompt problem (require a time reference in any inversion item), not a general quality problem, and it concentrates in the hard and medium bands where inversions appear.

5.4 Answer-position bias



Mean questions per plan whose correct answer sits at each authored slot, out of 30; a uniform plan would put 7.5 at each. The model is well balanced — 32/32 plans pass a χ^2 test against uniform, against 0/32 for the fixed authored bank. **This matters less than it looks.** The game has three lanes, not four, and `QuestionManager.toLaneChoices` trims every item at serve time by keeping the correct answer, dropping one random distractor, reshuffling the three, and recomputing the answer index. A correct answer authored at slot 3 is therefore never lost — it is re-indexed — and positional bias in the plan is neutralised before the player ever sees it. The measure is a signal about the model’s own generation habits, not a gameplay defect.

5.5 Player-facing language

MEASURE	MODEL PLAN	DETERMINISTIC
Reading grade (target ≈ 6)	7.13 ± 0.80	5.30 ± 0.38
Plans leaking a banned term	1/32	0/32
Plans using a discouraged term	0/32	0/32

The check runs on every string the player actually sees — feedback, curriculum titles, and each question’s explanation and reason. 1 of 32 plans matched a banned term, and the matches need reading before they are believed: `prompt` in `curriculum.sessionStrategy` — “...ward vocabulary. Keep the pace manageable but prompt the player to move faster to avoid running...”. That is the ordinary English verb *prompt* (“prompt the player to move faster”), not the machine-learning noun the rule was written to catch. The checker matches whole words but not word senses, so this is a **false positive** and the true leak rate for this run is 0 of 32. It is a good argument for keeping the excerpt in the output rather than just the count. Reading grade sits at 7.1 against a target of about 6, so the player-facing writing is roughly a year harder than intended — the one language result worth acting on.

5.6 Novelty

MEASURE	MODEL PLAN	DETERMINISTIC
Novel stems (not in authored bank)	99.1%	0.0%
Mean pairwise stem overlap	0.7%	100%
Max pairwise stem overlap	13.3%	100%
Unique stems / total generated	887 / 960	30 / 960

This is the axis the deterministic planner cannot compete on: it draws from a fixed 30-question bank, so every session is the same material. Pairwise overlap of 0.7% between generated plans means a player replaying a level mostly meets new questions.

5.7 By learner profile

An aggregate mean can hide the coach failing one kind of learner completely, which is why this table exists.

PROFILE	SERVED	1ST TRY	TARGET P	WEAK RECALL	DUE RECALL	ON CURRICULUM	GRADE
P1-cold-start	4/4	4	0.68	0.75	0.85	0.69	7.2
P2-struggling-beginner	4/4	4	0.30	0.19	0.85	0.63	6.3
P3-grammar-gaps	4/4	4	0.62	0.50	0.80	0.44	7.6
P4-advanced	4/4	4	0.49	0.63	1.00	0.73	7.8
P5-motor-misses	4/4	4	0.66	0.63	0.90	0.64	7.1
P6-early-puncher	4/4	4	0.53	0.31	0.95	0.57	6.7
P7-returning-decayed	4/4	4	0.69	0.88	1.00	0.91	7.7
P8-repeated-misconception	4/4	4	0.71	0.69	1.00	0.56	6.6

6 Engineering findings

Defects the evaluation surfaced before it could produce a single number. All four were fixed; the first two also affected the shipped game, not just the harness.

D1 The token-limit parameter is not portable across provider families

OpenRouter publishes a per-endpoint parameter list, and the request sets `require_parameters: true`, which discards any endpoint missing a parameter the request sends. The `openai/` namespace advertises `max_completion_tokens`; Google's endpoints advertise `max_tokens`. Because the code hardcoded `max_completion_tokens`, every request after the model switch was routed to zero endpoints and returned `404 No endpoints found that can handle the requested parameters`.

This was not confined to the evaluation: the game's own middleware sent the same parameter, so the in-game coach would have failed on every round and silently served the local fallback. Measured directly against the live API, the two families reject the opposite parameter, so a hardcoded name cannot be correct for both — the fix selects the parameter from the model slug.

D2 A 30-item array overflows Gemini's structured-output schema limit

With routing fixed, the provider began rejecting the request body itself with a bare `INVALID_ARGUMENT`. Bisecting the schema showed the cause is not a keyword but a size: Gemini expands a `minItems / maxItems` array into that many copies of the item schema, and 30 copies of an eleven-field question object exceeds its limit. Measured on the same model, eight items pass and sixteen fail, while a hundred copies of a one-field object pass — it is the expanded size that matters, not the count.

The fix drops length bounds only on large arrays of objects in the schema sent to the provider. Nothing is given up: the count is still stated in the prompt and still enforced on the response by the schema parse and by the difficulty-mix check, which is where a miscount was always caught.

D3 Provider errors were unreadable

The harness recorded only `"Provider returned error"` for every provider-side rejection, which is undiagnosable — a bad model slug, an unroutable provider filter and a rejected schema all looked identical. OpenRouter puts the upstream complaint in the error's `metadata.raw`; the harness now records it, which is how the schema limit above was found.

D4 The solver defaulted to the generator's own provider family

The independent check defaults its solver to a Google model. Once the generator became a Google model, that default would have had the family grade its own output, quietly converting the strongest evidence in the report into a measure of self-agreement. The solver is pinned to a different family for this run.

7 Threats to validity

- **Simulated learners, not real ones.** The reports are generated from behavioural profiles. They exercise the planner's input space, but they cannot show whether a plan actually helps a person learn — only whether it is well-formed, well-targeted against the evidence it was given, and sound as English.
- **The targeting metrics flatter the baseline.** The deterministic planner selects its target concepts directly from the weak and due lists in the report, so its precision and recall are 1.0 by construction rather than by judgement. The informative comparisons are the ones it cannot win by definition: item novelty, misconception quality, language, and whether questions sit on the curriculum.
- **One solver, one pass.** Agreement is evidence about an item, not proof. The authored-question control bounds the solver's own error rate, but disagreements still need adjudication by hand before they are called generator faults.
- **Concept names are matched as free text.** Targeting is scored with both exact and fuzzy matching because concept names travel from report to plan as strings; the fuzzy figure is the fairer one and the strict figure is reported beside it.
- **Transport resilience differs from the game.** A batch is 32 paid runs in one process, so the harness retries connection failures more times than the game does. This changes no generated token — it only stops a local network drop from voiding a paid batch.
- **Output headroom is thin.** Reasoning consumes 48% of the completion budget, leaving the visible plan to fit in what remains of the 18,000-token ceiling; mean completion is 13550 tokens (75% of the cap). A harder report could truncate rather than fail cleanly.

8 Reproducing this

Each phase writes its own JSON into `eval/results/`; the report is rendered from those files, so it cannot drift from the measurements it describes.

```
# free – check the simulated learners before spending anything
EVAL_PHASE=personas npx vitest run eval/

# billed – 32 sessions through the live planner
EVAL_PHASE=generate npx vitest run eval/

# billed – independent solver, pinned to a different provider family
EVAL_PHASE=judge EVAL_SOLVER_MODEL=openai/gpt-5.6-terra npx vitest run eval/

# free – deterministic baseline on the identical reports, then aggregate
EVAL_PHASE=baseline npx vitest run eval/
EVAL_PHASE=summarise npx vitest run eval/

# render from results/summary.json – all three read the same measurements
node eval/report.mjs          # this page
node eval/report-pdf.mjs      # PDF (prints this page)
node eval/report-doc.mjs      # Word
```

Punch English coach evaluation · planner google/gemini-3.1-pro-preview · solver openai/gpt-5.6-terra
· 32 sessions · rendered from eval/results/summary.json on 2026-09-04.